

SKILL WEEK 2

Session 3

Task 1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

int main() {
    char history[MAX_HISTORY][MAX_LENGTH];
    char input[MAX_LENGTH];
    int count = 0;
    int current = -1;

    printf("\033[1;32mSimple Command History\033[0m\n");
    printf("Type commands and use history navigation.\n");
    printf("Type 'exit' to stop.\n\n");

    while (1) {
        printf("shell> ");
        fgets(input, MAX_LENGTH, stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) > 0) {
            if (count < MAX_HISTORY) {
                strcpy(history[count], input);
                count++;
            } else {
                for (int i = 1; i < MAX_HISTORY; i++) {
                    strcpy(history[i - 1], history[i]);
                }
                strcpy(history[MAX_HISTORY - 1], input);
            }
        }
    }
}
```

```
GNU nano 7.2
while (1) {
    printf("shell> ");
    fgets(input, MAX_LENGTH, stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strlen(input) > 0) {
        if (count < MAX_HISTORY) {
            strcpy(history[count], input);
            count++;
        } else {
            for (int i = 1; i < MAX_HISTORY; i++) {
                strcpy(history[i - 1], history[i]);
            }
            strcpy(history[MAX_HISTORY - 1], input);

            current = count;

            printf("\033[1;34mCommand stored: %s\033[0m\n", input);
        }
    }

    printf("\n\033[1;33mCommand History:\033[0m\n");

    for (int i = 0; i < count; i++) {
        printf("%d: %s\n", i + 1, history[i]);
    }

    return 0;
}
```

```
ruhi0404@Ruhitha:~$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills3week2.c
ruhi0404@Ruhitha:~/Operating_Systems$
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills3week2.c -o skills3week2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills3week2
Simple Command History
Type commands and use history navigation.
Type 'exit' to stop.

shell> hello
Command stored: hello
shell> ruhitha
Command stored: ruhitha
shell>
shell> hello world
Command stored: hello world
shell> Operating_Systems
Command stored: Operating_Systems
shell> exit
Command stored: exit
shell> |
```

Observation

The program successfully implemented command history functionality by storing previously entered commands in a history buffer. Escape sequences were used for formatted terminal output, and the input buffer was updated whenever a new command was entered. The stored commands were displayed in order, demonstrating basic command recall and history management in a Linux environment.

Task 2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char data[100];
    struct Node *next;
};

int main() {
    struct Node *head = NULL, *temp, *newNode;
    char input[100];
    int count = 0;

    while (1) {
        printf("Enter data (type exit to stop): ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        newNode = (struct Node *)malloc(sizeof(struct Node));

        if (newNode == NULL) {
            printf("Memory allocation failed!\n");
            return 1;
        }

        strcpy(newNode->data, input);
        newNode->next = NULL;

        if (head == NULL)
            head = newNode;
        else {
            temp = head;

```

```

GNU nano 7.2
    }

    strcpy(newNode->data, input);
    newNode->next = NULL;

    if (head == NULL)
        head = newNode;
    else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }

    count++;

    printf("\nStored Data:\n");

    temp = head;
    while (temp != NULL) {
        printf("%s\n", temp->data);
        temp = temp->next;
    }

    temp = head;
    while (temp != NULL) {
        struct Node *nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }

    printf("\nMemory released successfully.\n");

    return 0;
}

```

```

6: nano skills3week2.c
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills3week2.c
ruhi0404@Ruhitha:~/Operating_Systems$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills3task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills3task2.c -o skills3task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills3task2
Enter data (type exit to stop): hello this is Operating system class
Enter data (type exit to stop): my name is ruhitha
Enter data (type exit to stop): exit
Enter data (type exit to stop): exit

Stored Data:
hello this is Operating system class
my name is ruhitha
exit

Memory released successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ valgrind --leak-check=full ./skills3task2
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully allocated memory dynamically using malloc() and stored user input in a linked list structure. Buffer overflow was prevented by limiting input size with fgets(). The linked list was managed correctly by adding and traversing nodes dynamically. All allocated memory was released using free(), and Valgrind verification confirmed that no memory leaks occurred. This experiment demonstrated proper dynamic memory management and safe handling of data structures in Linux.

SESSION 4

Task 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

struct Token {
    char value[50];
};

int main() {
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_TOKENS) {
        strcpy(tokens[count].value, token);
        count++;
        token = strtok(NULL, " \t\n");
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i].value);
    }

    if (count > 0)

```

```

    struct Token {
        char value[50];
    };

    int main() {
        char input[MAX_INPUT];
        struct Token tokens[MAX_TOKENS];
        int count = 0;

        printf("Enter a command: ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        char *token = strtok(input, " \t\n");

        while (token != NULL && count < MAX_TOKENS) {
            strcpy(tokens[count].value, token);
            count++;
            token = strtok(NULL, " \t\n");
        }

        printf("\nTokens:\n");

        for (int i = 0; i < count; i++) {
            printf("Token %d: %s\n", i + 1, tokens[i].value);
        }

        if (count > 0)
            printf("\nToken stream is valid.\n");
        else
            printf("\nNo tokens found. Invalid input.\n");

        return 0;
    }

```

```

^G Help
^X Exit

```

```

^O Write Out
^R Read File

```

```

^W Where Is
^_ Replace

```

```

^K Cut
^U Paste

```

```
ruhi0404@Ruhitha:~/Operating_Systems$ valgrind --leak-check=full ./skills3task2
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills4task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills4task1.c -o skills4task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills4task1
Enter a command: ls-1

Tokens:
Token 1: ls-1

Token stream is valid.
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills4task1
Enter a command: ls -l user

Tokens:
Token 1: ls
Token 2: -l
Token 3: user

Token stream is valid.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully split the user input into individual tokens using whitespace as delimiters. It handled spaces, tabs, and newline characters correctly and stored each token in a token structure. The generated token stream was validated and displayed for debugging, demonstrating basic input parsing and tokenization in a Linux environment.

Task 2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

struct Node {
    char value[50];
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(char *value) {
    struct Node *newNode = malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    strcpy(newNode->value, value);
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

void displayTree(struct Node *root, int level) {
    if (root == NULL)
        return;

    for (int i = 0; i < level; i++)
        printf("  ");

    printf("|-- %s\n", root->value);

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

```

        displayTree(root->left, level + 1);
        displayTree(root->right, level + 1);
    }

void freeTree(struct Node *root) {
    if (root == NULL)
        return;

    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command detected.\n");
        return 0;
    }

    /* Split input into tokens */
    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count++] = token;
        token = strtok(NULL, " \t");
    }
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```
GNU nano 2.9.2
}

/* Validate syntax */
if (count == 0) {
    printf("Syntax Error: No command entered.\n");
    return 0;
}

printf("\nSyntax is valid.\n");

/* Create parse tree */
struct Node *root = createNode("COMMAND");
root->left = createNode(tokens[0]);

if (count > 1) {
    struct Node *current = root->left;

    for (int i = 1; i < count; i++) {
        current->right = createNode(tokens[i]);
        current = current->right;
    }
}

printf("\nParse Tree:\n");
displayTree(root, 0);

printf("\nExecution Structure:\n");
for (int i = 0; i < count; i++) {
    printf("Argument %d: %s\n", i + 1, tokens[i]);
}

freeTree(root);
return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cu
```

```
ruhi0404@Ruhitha:~/Operating_Systems$ nano skills4task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills4task2.c -o skills4task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills4task2
Enter command: l -l ls user

Syntax is valid.

Parse Tree:
|-- COMMAND
  |-- l
    |-- -l
      |-- ls
        |-- user

Execution Structure:
Argument 1: l
Argument 2: -l
Argument 3: ls
Argument 4: user
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully designed basic parser logic by splitting the input into tokens and validating the command syntax. A parse tree was generated to represent the command structure, and syntax errors and empty commands were detected and handled. The program also produced an execution structure showing the command and its arguments, demonstrating the basic working of a command parser.

